

NEUROPLAY

AI-Driven EMG Gaming for Therapy



By
Waleed Khan B22F0611SE113
Muhammad Azeem B22F0654SE032
Haris Faisal B22F0709SE047

Final Year Project SRS

School of Computing Sciences
(Software Engineering)

**Pak-Austria Fachhochschule: Institute of Applied
Sciences & Technology**

2022 - 2026



PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

NeuroPlay:AI-Driven EMG Gaming For Therapy

A Final Year Project Report Presented to
Pak-Austria Fachhochschule: Institute of Applied Sciences and
Technology

In partial fulfillment of the
requirement for the degree of
BS Software Engineering

By
Waleed Khan
B22F0611SE113

Muhammad Azeem Sikandar
B22F0654SE032

Haris Faisal
B22F0709SE047

2022 - 2026

NeuroPlay: AI-Drive EMG Gaming for therapy

A Final Year Project Report submitted to the School of Computing Sciences as partial fulfillment of the requirement for the award of Degree of BS in Software Engineering.

Waleed Khan	B22F0611SE113
Muhammad Azeem Sikandar	B22F0654SE032
Haris Faisal	B22F0709SE047

Academic Supervisor

Dr. Musadaq Mansoor
Assistant Professor,
School of Computing Sciences ,
Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology

Final Approval

This final year project titles

NeuroPlay:

By

<i>Waleed Khan</i>	<i>B22F0611SE113</i>
<i>Muhammad Azeem Sikandar</i>	<i>B22F0654SE032</i>
<i>Haris Faisal</i>	<i>B22F0709SE047</i>

under the supervision of their project supervisor and approved by the project evaluation committee, has been accepted by the Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology (PAFIASST), Pakistan, in partial fulfillment of the requirements for the degree of BS Software Engineering.

Academic Supervisor:

Dr.Musadaq Mansoor
Assistant Professor
School of Computing Sciences
PAF-IASST ,KP,Pakistan.

HoD / Chairman:

Dr.Kamran Javed
Associate Professor
School of and Computing Sciences,
PAF-IASST ,KP,Pakistan.

Declaration

We, Waleed Khan, B22F0611SE113, M.Azeem Sikandar, B22F0654SE032 and Haris Faisal, B22F0709SE047, hereby declare that we have produced the work presented in this final year project report, during the scheduled period of study. We also declare that we have not taken any material from any source except referred to wherever due to that amount of plagiarism is within an acceptable range. It is further declared that we have developed this project and the accompanied report entirely on the basis of our personal efforts made under the sincere guidance of our supervisor. No portion of the work presented in this report has been submitted in support of any other degree or qualification of this or any other

University or Institute of learning, if found we shall stand responsible.

Date: _____

Signature: _____

Waleed Khan

B22F0611SE113

Signature: _____

Muhammad Azeem Sikandar

B22F0654SE032

Signature: _____

Haris Faisal

B22F0709SE047

Certificate

It is certified that Waleed Khan B222F0611SE113, Muhammad Azeem Sikandar B22F0654SE032 & Haris Faisal B22F0709SE047 have carried out all the work related to this project under my supervision at the Department of IT and Computer Science, Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology (PAF-IAST) and the work fulfills the requirement for the award of BS Software Engineering.

Date: _____

Supervisor:

Dr. Musadaq Mansoor
Assistant Professor
School of Computing Sciences,
PAF-IAST, KP, Pakistan.

Head of Department:

Dr. Kamran Javed
Associate Professor
School of Computing Sciences, PAF-IAST,
KP, Pakistan.

DEDICATION

This project is dedicated to all those who have provided their unwavering support and guidance throughout this journey. To our parents, for their endless love, encouragement, and belief in my abilities. Your sacrifices and constant motivation have been my driving force. To our project supervisor, Dr. Musadaq Mansoor, whose expert advice, insightful feedback, and continuous support were invaluable to the success of this project. We are indeed very grateful for your support throughout this process since it has been very helpful. To our friends and fellow workers to whom we owe time, advice and help offered, encouragement given. Your support and encouragement help us with problems and success. To all the talented web designers, developers and other clients who have taken your precious time to contribute to the development as well as the testing process of NeuroPlay, your contribution is greatly appreciated in the making of this project. Last but not least, we would like to extend our sincere appreciation to all who encouraged us and this work, you are the reason we woke up every morning to continue with this task. We suppose each of us has felt overwhelmed to be a part of this team and contribute to the achievement of this project.

ACKNOWLEDGMENTS

We would like to extend our heartfelt thanks to our respected supervisor Dr. Musadaq Mansoor for his continuous guidance, encouragement, and support throughout the completion of this project. His insightful feedback and mentorship had a very crucial role in achieving the project's objectives and direction.

It has truly been an honor to work under the academic supervision of Dr. Musadaq Mansoor. This collaboration has provided a valuable opportunity to apply theoretical concepts to practical challenges and gain hands-on experience in the field.

ABSTRACT

Neuroplay: AI-Driven EMG Gaming for Therapy

With the rapid advancement in Artificial Intelligence and biomedical signal processing, innovative approaches are being developed to enhance human–computer interaction. Traditional game control systems rely on physical devices such as keyboards, joysticks, or sensors, limiting accessibility for users with physical impairments. The proposed system aims to introduce an AI-driven EMG-based control interface that enables players to interact with digital games using their muscle signals instead of conventional input devices.

The system captures electromyography (EMG) signals generated by muscle contractions through a MyoWare EMG sensor. The continuous EMG data is sent to a Python-based AI module, where it is preprocessed, normalized, and passed into a trained neural network model that classifies distinct muscle movements such as bicep flexing or relaxation. Based on the model’s prediction, a corresponding control signal (e.g., move left or right) is transmitted to the game through web socket.

The AI model is trained on a custom EMG dataset collected under varying muscle contraction intensities to ensure consistent recognition across both slow and fast movements. The processed output enables the web socket to act as an intelligent bridge between human muscle activity and the game’s control interface, providing real-time response and smooth character movement.

Through the integration of AI, embedded hardware, and human physiology, the system provides an intelligent, hands-free gaming solution. This project not only enhances gaming experiences but also demonstrates the potential of EMG-based interfaces for rehabilitation, assistive technologies, and biomedical applications.

Table of Contents

Chapter 1.....	13
Project Vision.....	13
1.1 Problem Statement.....	13
1.2 Business Opportunity.....	13
1.3 Objectives.....	13
1.4 Project Scope.....	14
1.5 Constraints.....	14
1.6 Feasibility.....	15
1.7 Stakeholder Description.....	16
1.7.1 Stakeholders Summary.....	16-17
1.7.2 Key High-Level Goals and Problems of Stakeholders.....	17
Chapter 2.....	18
Software Requirement Specification.....	18
2.1 List of Features.....	18
2.2 Functional Requirements.....	19
2.3 Non-Functional Requirements.....	19
2.4 Use Cases/ Use Case Diagram.....	20
2.4.1	20
2.4.2.....	21
2.4.3.....	21
2.4.4.....	22
Chapter 3	24
System Overview.....	24
3.1 Architectural Design.....	24
3.2 Data Design.....	25
3.2.1 Entity-Relationship Diagram (ERD).....	25
3.2.2 Data Flow Diagram (DFD).....	26
3.3 Domain Model.....	27
3.4 UML Diagrams.....	27
3.4.1 Activity Diagram.....	28
3.4.2 Class Diagram	29
3.4.3 Class-level Sequence Diagram	30
3.4.4 State Transition Diagram.....	31
3.4.5 Sequence Diagram	32
3.4.7 Component Diagram	33
3.5 Story Line.....	33

Chapter 4	34
System Implementation.....	34
4.1 Technology Stack (Languages, Frameworks, Tools)	34
4.2 Development Environment Setup.....	34
4.3 System Modules and Components.....	34
4.4 Dataset Selection and Preprocessing.....	35
4.5 Model Selection and Training.....	35
4.6 Model Evaluation.....	35
4.7 Model Deployment.....	35
Chapter 5	36
System Testing & Deployment.....	36
5.1 Testing Approach (Manual, Automated).....	36
5.2 Unit Testing	36
5.3 Integration Testing.....	36
5.4 User Acceptance Testing (UAT)	36
5.5 Performance Testing.....	37
5.6 Deployment Strategy	37
5.7 Hosting & Deployment Environment.....	37
Chapter 6	38
Results and Discussion.....	38
6.1 Evaluation and Results.....	38
6.1.1 Comparison with Initial Objectives.....	38
6.1.2 User Feedback and Usability Testing.....	38
6.1.3 Performance Evaluation.....	38
6.1.4 Challenges and Limitations.....	39
Chapter 7	40
Conclusion and Future Work	40
7.1 Conclusion	40
7.2 Future Enhancements.....	40
BIBLIOGRAPHY.....	41

List of Figures

Fig 2.4 Use Case Diagram.....	20
Fig 3.1 Architectural Design.....	24
Fig 3.2.1 ERD.....	25
Fig 3.2.2 DFD.....	26
Fig 3.3 Domain Model.....	27
Fig 3.4.1 Activity Diagram.....	28
Fig 3.4.2 Class Diagram.....	29
Fig 3.4.3 Class-level Sequence Diagram.....	30
Fig 3.4.4 State Transition Diagram.....	31
Fig 3.4.5 Sequence Diagram.....	32
Fig 3.4.6 Component Diagram.....	33
Table 5.2 Unit Testing.....	36
Table 5.3 Integration Testing.....	36
Table 5.4 User Acceptance Testing (UAT).....	36
Table 5.5 Performance Testing.....	37

Chapter 1

Project Vision

1.1 Problem Statement

Stroke patients and people with partial paralysis face difficulty in performing rehabilitation exercises due to their limited muscle movement. Traditional therapy methods are often repetitive and uninteresting, which reduces patient motivation and recovery progress. In addition, normal game controllers are not suitable for such patients because they require fine hand control and quick movements. This creates a need for a more engaging and accessible way to help these patients in their rehabilitation process.

1.2 Business Opportunity

This project has a great business opportunity in the area of healthcare and rehabilitation. Stroke patients and people with partial paralysis often find traditional therapy slow, repetitive, and boring. Our system offers a new way to make therapy more interactive and enjoyable by using AI and EMG signals to control simple games through muscle movement.

This idea can be turned into a real product that helps patients exercise and recover quickly without needing constant supervision. Physiotherapy centers and rehabilitation clinics can also use it to make their sessions more engaging and effective. In the future, this system can be developed into a wearable or mobile-based device, making it easier for patients to use anywhere. It also opens doors for collaboration with healthcare companies, game developers, and medical startups. Overall, it has strong potential to grow as an affordable and helpful solution for people with movement disabilities.

1.3 Objectives

The primary objectives of this project are as follows:

1. To develop an AI model that processes EMG signals to recognize different muscle movements.
2. To assist stroke patients and people with partial paralysis in performing interactive rehabilitation exercises.
3. To connect the AI model with the Web socket for controlling game movements based on muscle activity.
4. To make therapy more engaging and effective through game-based muscle training.
5. To create an affordable and accessible rehabilitation system for physiotherapy.

1.4 Project Scope

1.4.1 In Scope:

- I. Designing a system that captures muscle signals using an EMG sensor.
- II. Developing an AI model to classify different muscle movement patterns.
- III. Interfacing the AI module with a web socket which will act as a bridge to control a game based on detected muscle activity.
- IV. Creating an interactive game environment to make rehabilitation exercises engaging.

1.4.2 Out of Scope:

- I. Medical diagnosis or treatment of patients' health conditions.
- II. Full-scale hardware integration for wearable or commercial use.
- III. Real-time cloud data storage or remote therapy monitoring.
- IV. Support for multiple sensors or complex body movement recognition.
- V. Long term system updates and maintenance.

1.5 Constraints

a) Hardware Sensitivity:

The accuracy of the system strongly depends on the EMG sensor's quality, placement, and calibration. Any disturbance like loose electrodes, body movement, or sweat can introduce noise and reduce signal reliability.

b) User Variability:

EMG signals differ greatly between individuals based on age, muscle condition, skin texture, or fatigue level. The system may require re-calibration for each new user to achieve consistent performance.

c) Limited Movement Detection:

The system currently supports only a few specific muscle actions (e.g., left, right). Complex gestures or multi-muscle coordination are not yet supported.

d) Environmental Interference:

External electrical devices, temperature changes, and environmental noise can affect EMG readings, leading to false classifications or unstable responses.

e) Training Complexity:

Developing and training the AI model requires careful preprocessing, normalization, and labeling of EMG signals, which can be time-consuming and computationally demanding.

f) No Medical Certification:

The prototype is designed for research and educational purposes. It is not medically approved for clinical rehabilitation or diagnosis.

g) User Safety and Fatigue:

Prolonged or repeated muscle activation during testing may cause discomfort or strain, especially for patients with limited mobility, requiring supervision and safety consideration.

1.6 Feasibility

- **Technical Feasibility:**

The project is technically feasible because it uses readily available and low-cost hardware such as the MyoWare EMG sensor and a computer or laptop for AI processing. Python libraries like TensorFlow or PyTorch can be used for building and training the AI model, while web socket act as a bridge between the model and the game allows real-time data exchange between the AI model and game. The technologies involved are open-source and well-documented, making development achievable for a small research team.

- **Economic Feasibility:**

The system is cost-effective compared to existing rehabilitation machines and robotic therapy devices, which are often expensive and inaccessible for patients in developing countries. The hardware components (EMG sensor and basic wiring) are affordable, and the software tools used are free or open-source. This makes the project financially viable for both development and future commercialization as a low-cost rehabilitation solution for hospitals, clinics, and home therapy setups.

- **Operational Feasibility**

The system can easily be operated by patients with minimal training. Once the device is calibrated, the user only needs to perform simple muscle movements to interact with the game. The real-time response of the AI model ensures smooth and engaging gameplay. Physiotherapists can also use the system to monitor progress or adapt exercises. Hence, the system is practically usable and beneficial in rehabilitation sessions.

- **Social Feasibility**

This project promotes inclusivity and accessibility by helping stroke patients and people with partial paralysis regain confidence and mobility through fun, game-based therapy. It reduces dependency on traditional therapy routines, encourages motivation, and improves emotional well-being. The idea supports social impact by combining healthcare and technology to improve quality of life.

1.7 Stakeholder Description

1.7.1 Stakeholders Summary

There are the following key stakeholders involved in this project:

1. End Users (Primary Stakeholders):

- **Individuals:** The primary target users include stroke patients who need physiotherapy , making it easier and more engaging for them through gaming approach.
- **Healthcare Providers:** Doctors and physiotherapists will benefit from the system as it provides gaming based physiotherapy for patients, improving recovery instead of the traditional methods.
- **Healthcare Institutions:** Hospitals and clinics will be key stakeholders, as they can adopt the system for the therapy of the stroke patients.

2.University (Pak-Austria Fachhochschule):

The university, as the project sponsor, provides academic oversight and the necessary resources for the project. Professors and the university's research community are also stakeholders, as they may use the system for research purposes, healthcare studies, or as a teaching tool for future students.

3.Potential Investors or Funding Bodies:

If the project expands beyond the academic phase, investors and funding agencies will become stakeholders. They will be interested in the project's potential for commercialization, especially in the healthcare sector, and its scalability.

4.Development Team:

- **The Developers:** The developers, as the project leads, are responsible for planning, developing, testing, and implementing the system. They are tasked with ensuring that the system meets all objectives and performs accurately in real-world scenarios.
- **Technical Advisors:** Any technical experts or consultants who may assist with overcoming specific challenges, such as enhancing the model or improving system performance, are important stakeholders.

5.Academic and Industrial Supervisors:

The academic and industrial supervisors play a crucial role by providing guidance, feedback, and oversight throughout the project. They ensure that the project meets academic standards and aligns with research goals, offering valuable insights into its technical and practical application in healthcare settings.

1.7.2 Key High-Level Goals and Problems of Stakeholders

The development of NeuroPlay aims to address critical challenges faced by multiple stakeholders in the rehabilitation and healthcare ecosystem, particularly stroke survivors, physiotherapists, and healthcare providers. One of the primary goals is to support patients with limited motor control by providing an intelligent therapy assistance system that responds to muscle activity detected through EMG signals. Stroke and partially paralyzed patients often struggle with slow or weak muscle contractions, making it difficult to perform consistent and effective physical therapy sessions without continuous supervision.

For physiotherapists, managing multiple patients and ensuring each receives adequate, personalized therapy time is a major challenge. Manual observation of progress can be inaccurate and time-consuming. The system aims to automate therapy movement detection and control, reducing the therapist's workload while maintaining accurate progress tracking. Healthcare providers and rehabilitation centers often face a shortage of affordable, adaptive therapy technologies. Existing robotic or sensor-based rehabilitation systems are either too expensive or technically complex for widespread use. This project seeks to offer a cost-effective and accessible solution, enabling wider adoption in low-resource medical environments.

By leveraging machine learning and real-time EMG signal processing, the proposed system provides a smart, safe, and automated way to assist in muscle rehabilitation. It empowers patients to perform therapy independently, while still allowing medical professionals to monitor performance and progress remotely, thus improving efficiency, accuracy, and recovery outcomes in post-stroke rehabilitation.

Chapter 2

Software Requirement Specification

2.1 List of Features

- **Real-Time EMG Signal Acquisition:**

Continuously captures muscle activity signals from the patient's bicep using EMG sensors.

- **AI-Based Movement Detection:**

Uses machine learning to analyze EMG signals and identify muscle movements such as contraction and relaxation.

- **Decision-Making Module:**

Processes EMG signal data and determines the appropriate command (e.g., move left ,right, etc.) for the connected system.

- **Web Socket :**

Sends control signals from the AI model to unity interface for executing corresponding actions in the external system or game.

- **Rehabilitation Game Integration:**

Allows patients to control a simple game using their muscle movements, making physical therapy engaging and interactive.

- **Progress Tracking:**

Monitors patient performance to help track rehabilitation progress over time.

- **Safe and Non-Invasive Operation:**

Ensures patient safety by using non-invasive sensors and controlled response mechanisms.

2.2 Functional Requirements

1. The system should continuously capture EMG signals from the patient's bicep using EMG sensors.
2. The system should send real-time EMG data to the AI model for processing and classification.
3. The AI model should detect and differentiate between muscle contractions, relaxations, and idle states.
4. The system should process the EMG signals and decide which command (left, right) to send to game through web socket .
5. The game should receive the AI model's output and send the correct signal to the connected game.

6. The system should allow patients to play and control a rehabilitation game using only their muscle movements.
7. The game should respond immediately to the patient's movement commands for real-time control.
8. The system should show connection status between EMG sensors, AI module, and web socket to avoid confusion.
9. The interface should be easy enough for patients with limited mobility to operate independently.
10. The therapist should be able to monitor and analyze the patient's improvement over multiple sessions.

2.3 Non-Functional Requirements

1. The system should process and send signals in real-time or near real-time with less than one-second delay.
2. The AI model should maintain high accuracy (around 85% & above) in classifying muscle movements.
3. The system should remain stable and reliable during continuous use without frequent crashes or lags.
4. The interface should be clean, simple, and easy for both patients and therapists to understand.
5. The system should use safe, non-invasive EMG sensors that cause no discomfort or harm.
6. The software should be designed so it can be easily updated or maintained by developers.
7. The AI model should be flexible enough to include more movements or sensors in future versions.
8. The system should operate effectively even with small noise or signal variation in EMG readings.
9. The game should run smoothly and provide quick feedback to make the therapy engaging.
10. The user interface should support accessibility features like large buttons and clear visuals.

2.4 Use Cases/ Use Case Diagram

Use Case Diagram:

The use case diagram in Fig 2.4 visually represents the key functionalities of the system from the user's perspective. It outlines the interactions between the main actors (User and System) and the system's use cases, such as *Emg signals*, *AI Model*, *Web socket functionalities* and *game connectivity*. Each use case encapsulates a specific goal or task that the actor wants to accomplish. Solid lines indicate associations between

actors and use cases, while the diagram helps in understanding system behavior, user roles, and functional requirements at a high level.

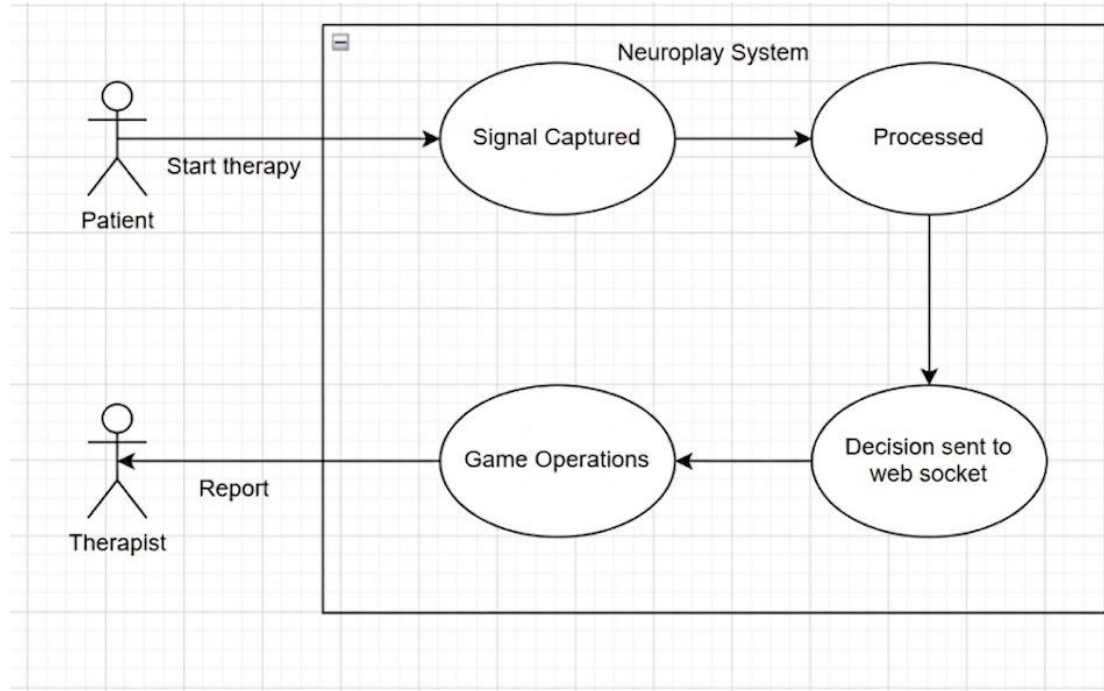


Fig 2.4: Use Case Diagram

Use Case 1: Start Rehabilitation session

Table 2.4.1: Scan Prescription

Attribute	Description
Use Case Name	Start Rehabilitation session
Actor	User (Patient)
Description	The User starts a rehabilitation game session where muscle signals are used to control game movements.
Preconditions	EMG sensors are properly connected to the game through web socket; the system is powered on.
Basic Flow	1. Patient opens the application. 2. Selects "Start Session." 3. System begins reading EMG signals in real-time. 4. Game starts responding to patient's muscle movements.

Alternative Flow	2a. Sensor not connected → Show “Sensor Not Found” message and guide user to reconnect.
Postconditions	The session begins, and real-time EMG data is processed for controlling the game.

Use Case 2: Detect and Classify Muscle Movement

Table 2.4.2: Refine Extracted Text

Attribute	Description
Use Case Name	Detect and Classify Muscle Movement
Actor	AI Model
Description	The AI model processes EMG signals and classifies muscle movements (e.g., contraction, relaxation).
Preconditions	.EMG signals are being received from the sensor.
Basic Flow	1. System receives raw EMG data. 2. AI model pre processes and normalizes the signal. 3. Model classifies the movement type. 4. Decision is sent for execution.
Alternative Flow	3a. Signal noise detected → Movement not classified.
Postconditions	Movement is classified and corresponding command is generated.

Use Case 3: Send Command to Game

Table 2.4.3: Manage Prescriptions

Attribute	Description
Use Case Name	Send Command to Game Act as a bridge between model and the game
Actor	Web socket
Description	Web socket receives movement commands from the AI model and sends them to the game for execution.
Preconditions	AI model has processed a valid EMG signal
Basic Flow	1. AI model sends command to the web socket (e.g., left, right). 2. Web socket transmits signal to the game interface. 3. Game character moves accordingly.

Alternative Flow	2a. Communication failure → game displays “Connection Lost.”
Postconditions	The game responds to the patient’s muscle movement command.

Use Case 4: View Progress Report

Table 2.4.4: User Profile Management

Attribute	Description
Use Case Name	View Progress Report
Actor	Therapist
Description	Therapist views a patient’s performance data to assess recovery progress
Preconditions	Patient has completed one or more therapy sessions.
Basic Flow	1. Therapist logs into the system. 2. Navigates to “Progress Report.” 3. Selects a patient’s profile. 4. System displays detailed performance metrics (accuracy, reaction time, improvement trend).
Alternative Flow	3a. No data available → Show “No session data found.”
Postconditions	Therapist reviews and analyzes the patient’s rehabilitation progress.

Chapter 3

System Overview

3.1 Architectural Design

The system follows a layered architecture including the User Interface, Application Logic, and AI Signal Processing layers. EMG signals captured using the MyoWare sensor are processed by a Python-based AI module to classify muscle movements. The predicted actions are sent to the web socket, which sends them to game, enabling hands-free interaction

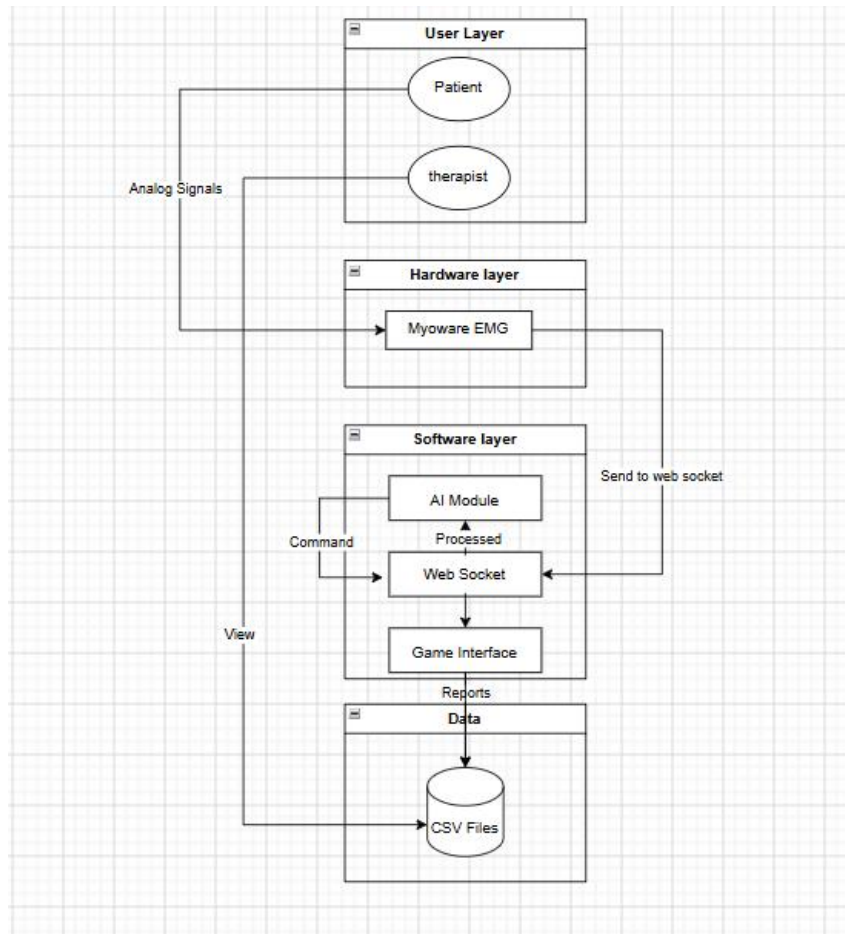


Fig 3.1: Architectural Design

3.2 Data Design

3.2.1 Entity-Relationship Diagram (ERD)

The ERD represents the data structure of the system by defining entities such as Patient, Therapy Session, EMG Data, and Game Profile, along with their relationships. Each entity includes essential attributes (e.g., Patient ID, Session Date, EMG Signal Values, Game Score). The relationships illustrate how a patient can have multiple therapy sessions, and each session can store associated EMG signals and performance data. This diagram supports the design of an organized and efficient database for tracking therapy progress.

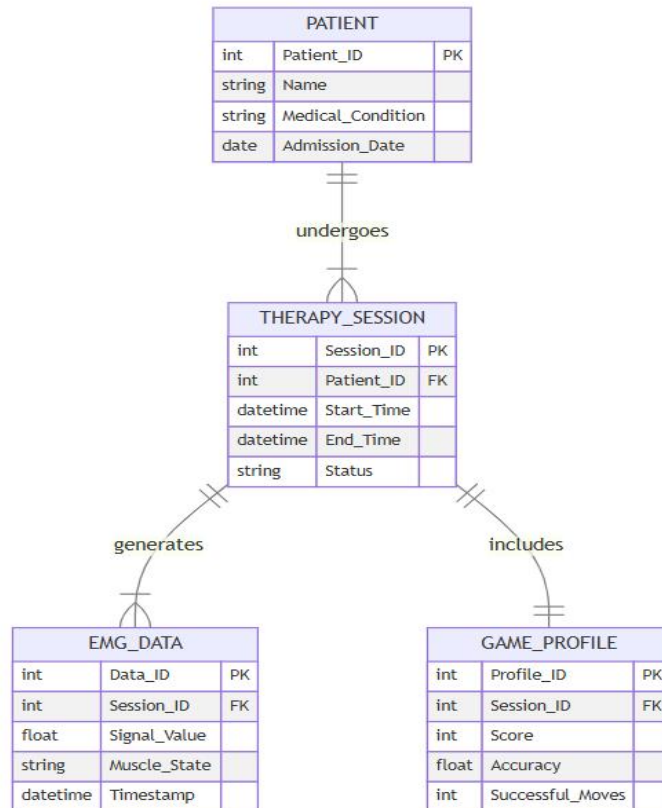


Fig 3.2.1: ERD

3.2.2 Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves through the system during operation. It shows the flow of EMG signals from the MyoWare sensor to the AI processing module, where the signals are preprocessed and classified. The predicted movement commands are then sent to the web socket and forwarded to the game interface. Therapy and performance data are stored for future analysis, providing a clear view of system inputs, processing stages, outputs, and data storage.

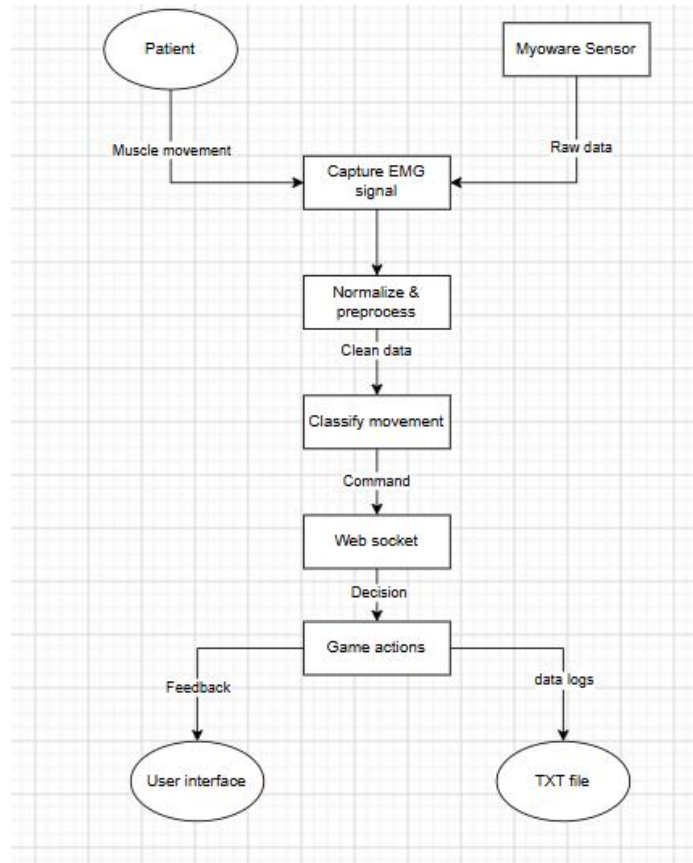
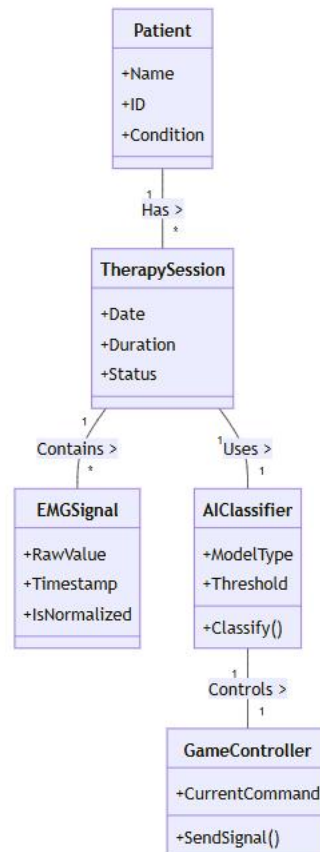


Fig 3.2.2: DFD

3.3 Domain Model

The domain model defines the core concepts and their relationships within the system. It includes key classes such as Patient, Therapy Session, EMG Signal, AI Classifier, and Game Controller, along with their attributes and interactions. This model provides a conceptual blueprint of how muscle signals are captured, processed, classified, and translated into game actions, guiding the overall software design and development.

Fig 3.3: Domain Model



3.4 UML Diagrams

3.4.1 Activity Diagram

The activity diagram represents the step-by-step workflow of the system, from capturing EMG signals to processing them using the AI model and generating game control actions. It highlights the sequence of operations, decision points, and user interaction during a therapy or gaming session, providing a clear view of system behavior.

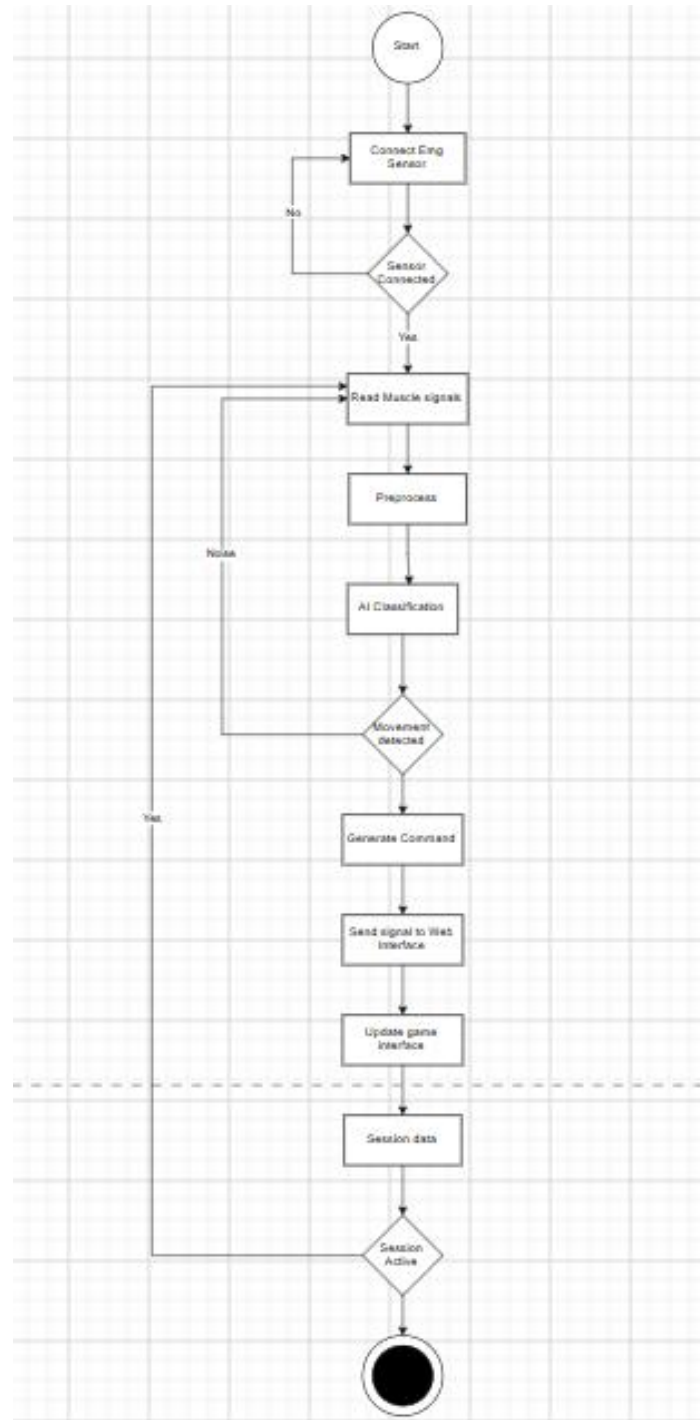


Fig 3.4.1: Activity Diagram

3.4.2 Class Diagram

The class diagram illustrates the static structure of the system by defining classes such as **Patient**, **Therapy Session**, **EMG Signal**, **AI Model**, and **Game Controller**, along with their attributes and methods. It shows how these classes interact to process muscle signals and generate game control actions, supporting the system's overall functionality.

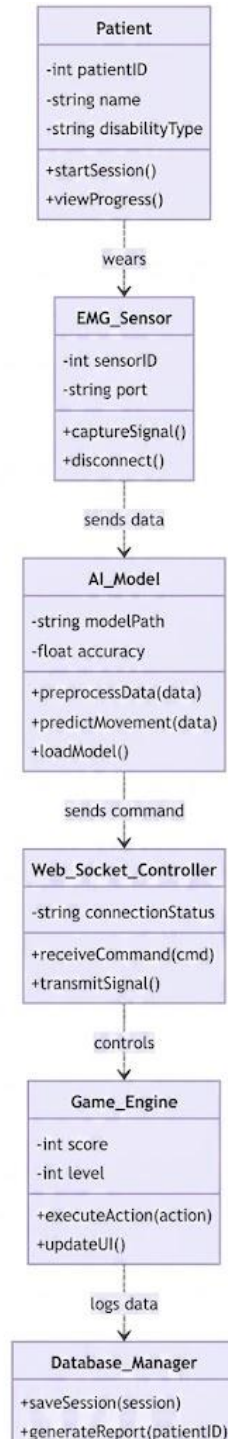


Fig 3.4.2: Class Diagram

3.4.3 Class-level Sequence Diagram

The class-level sequence diagram illustrates how system components interact over time during a therapy or gaming session. It shows the sequence of interactions between classes such as EMG Sensor, AI Processor, web socket, and Game Interface, demonstrating how muscle signals are processed and converted into real-time game actions.

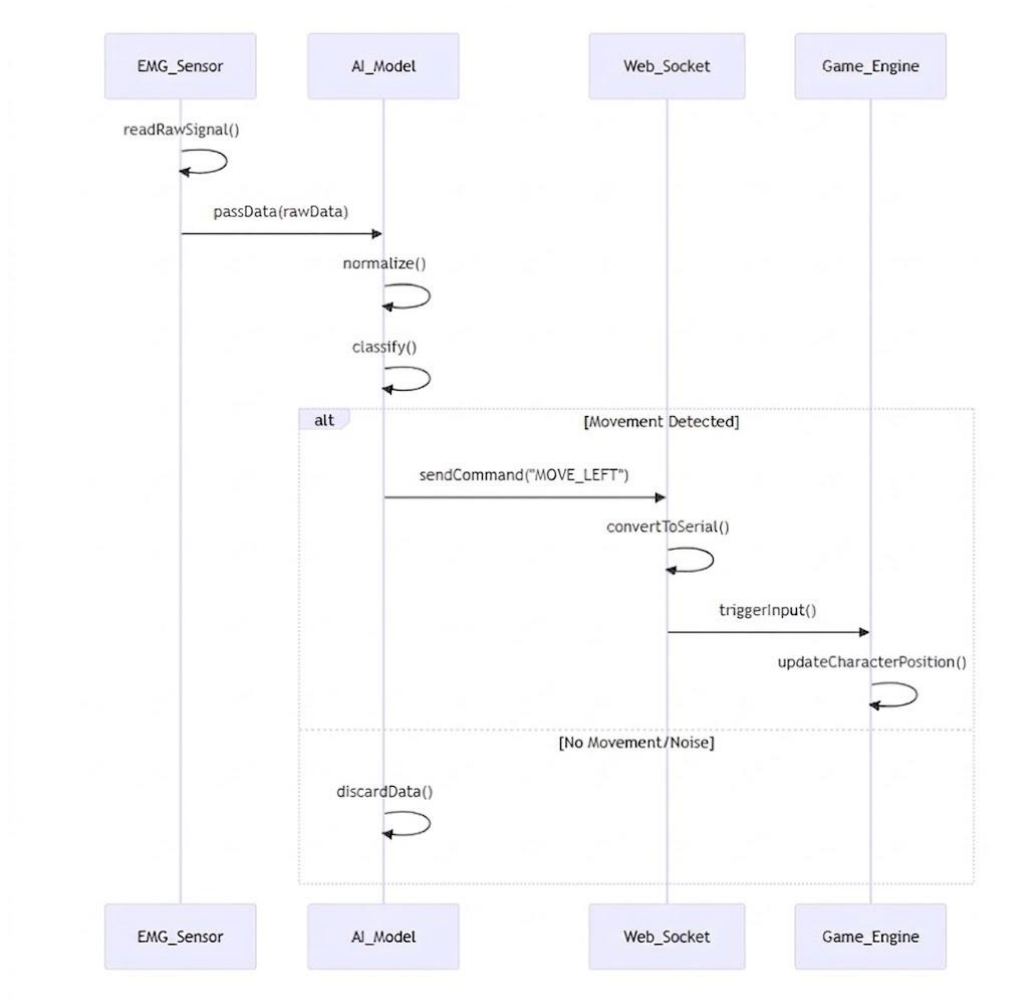


Fig 3.4.3: Class-level Sequence Diagram

3.4.4 State Transition Diagram

The state transition diagram represents the different states of a therapy session or game control process, such as Idle, Signal Capture, AI Processing, Action Execution, and Session End. It illustrates how the system transitions between states based on user actions and muscle activity, helping to understand the dynamic behavior of the system.

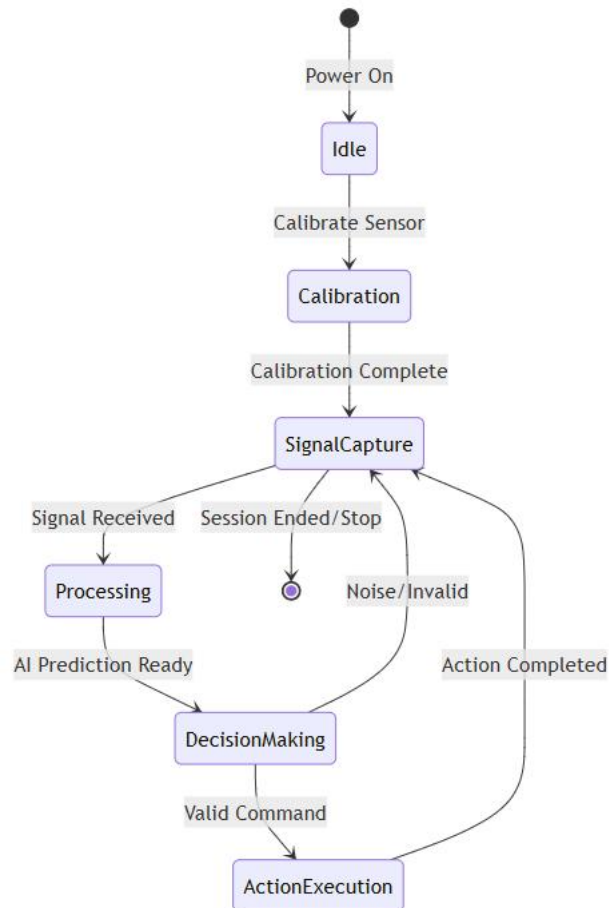


Fig 3.4.4: State Transition Diagram

3.4.5 Sequence Diagram

The sequence diagram shows the chronological interaction between system components during a therapy or gaming session. It illustrates the exchange of messages from EMG signal capture to AI classification and game control execution, explaining how system components coordinate in real time.

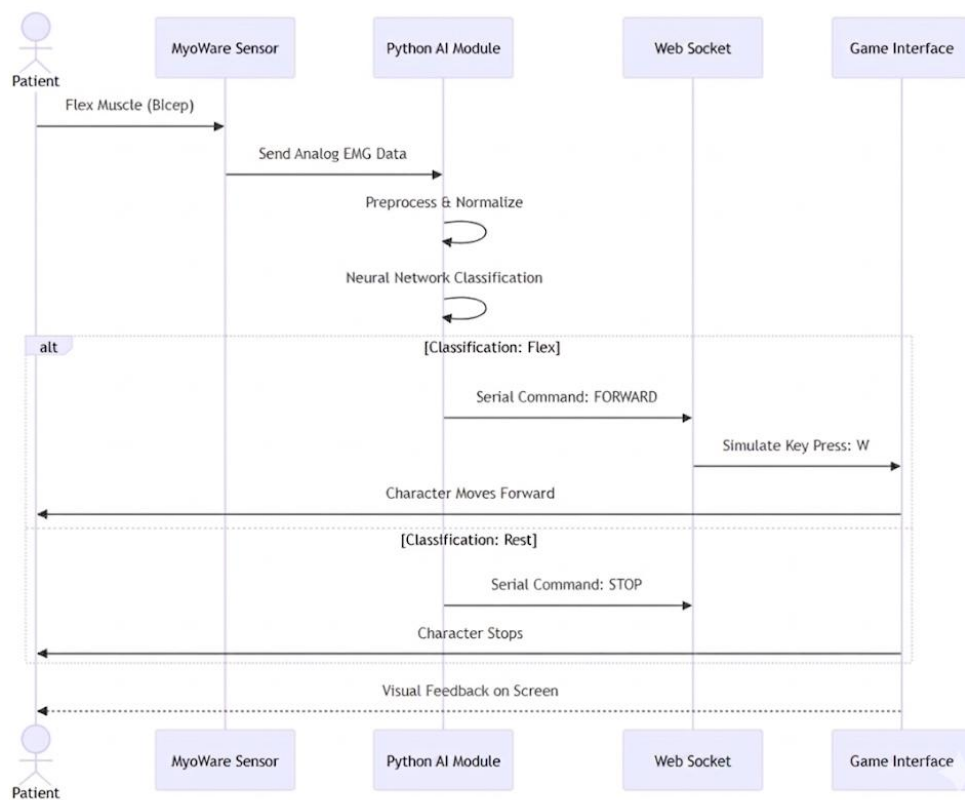


Fig 3.4.5: Sequence Diagram

3.4.6 Component Diagram

The component diagram shows the high-level structure and dependencies of the system, including components such as EMG Sensor Interface, AI Processing Module, Web socket, Game Interface, and Data Storage. It illustrates how these modules interact and integrate to enable real-time, hands-free gaming and therapy tracking.

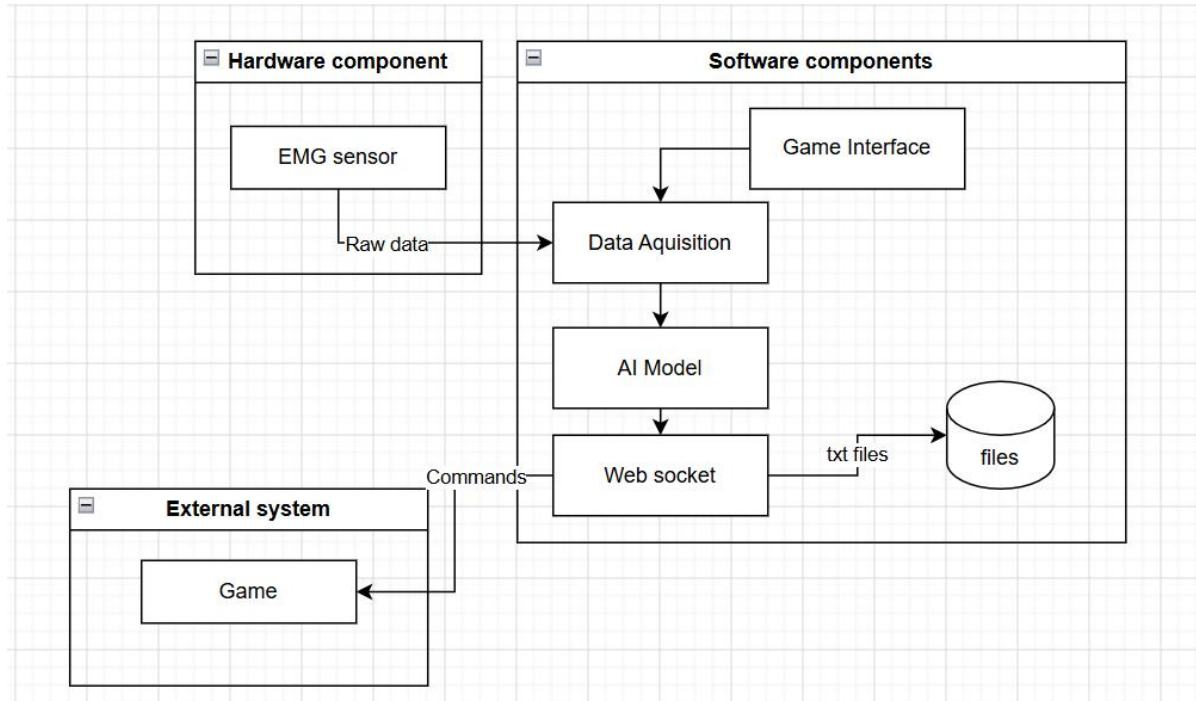


Fig 3.4.6: Component Diagram

3.4 Story Line

The game is developed as part of the *NeuroPlay* system to support interactive motor training through EMG-based input. In the game, the player controls a ball moving through a predefined pathway. The ball represents processed muscle signals that are converted into real-time movement within the game environment.

The pathway contains rectangular obstacles that the ball must avoid to successfully complete each level. The ball moves left or right based on detected muscle activity, requiring controlled and precise input from the user. This interaction promotes coordination and stability during gameplay.

The difficulty of the game increases gradually by adding more obstacles and reducing available space, reflecting progressive training levels. The game transforms muscle-based control into an engaging experience, aligning therapeutic goals with interactive gameplay.

Chapter 4

System Implementation

4.1 Technology Stack (Languages, Frameworks, Tools)

1. **Programming Languages:**

- Python (for machine learning models and backend development)
- C#(for game in unity & web socket)
- python(for API Management and Web socket)

2. **Frameworks:**

- Unity Engine (for developing the interactive rehabilitation game)
- Arduino Platform (for EMG signal acquisition and communication)

3. **Libraries and Tools:**

- NumPy / Pandas (for data handling and preprocessing)
- Scikit-learn / TensorFlow (used for machine learning model implementation and training)
- MyoWare EMG Sensor (for capturing muscle signals)
- Git & GitHub for version control

4.2 Development Environment Setup

To implement the system, the development environment was set up as follows:

- **AI and Signal Processing:**

Python environment (3.14+) was used for EMG signal processing, data handling, and model execution.

- **Hardware Integration:**

Web socket were used to interface the MyoWare EMG sensor with the game , enabling real-time signal acquisition.

- **Game Development:**

Unity Engine was used to design the game environment, where player movement is controlled through EMG-based inputs.

- **Data Handling:**

EMG signals and session data were stored in structured formats such as CSV files for analysis and model training.

4.3 System Modules and Components

The system is divided into several key modules:

- **EMG Signal Acquisition Module:**

Captures muscle activity from biceps and triceps using the MyoWare sensor and sends signals to the Web socket .

- **Signal Processing Module:**

Processes raw EMG signals by applying normalization and basic filtering techniques.

- **AI Classification Module:**

Uses a machine learning model to classify muscle movements such as flexion and relaxation.

- **Game Control Module:**

Converts classified outputs into control commands for the Unity-based game.

4.4 Dataset Selection and Preprocessing

A custom dataset was generated using real-time EMG signals collected from the MyoWare sensor placed on biceps and triceps muscles.

Preprocessing steps include:

- **Signal Normalization:**
Standardizing EMG values for consistent model input
- **Noise Reduction:**
Applying smoothing techniques to reduce signal fluctuations
- **Segmentation:**
Dividing continuous signals into smaller segments for classification
- **Data Labeling:**
Assigning labels such as flexion and relaxation to each signal segment

4.5 Model Selection and Training

A machine learning model was used to classify EMG signals into different muscle movements.

- The model was selected based on its ability to handle time-series signal data
- The dataset was divided into training and testing sets
- The model was trained to recognize patterns in EMG signals
- Basic tuning was performed to improve classification performance

4.6 Model Evaluation

The model performance was evaluated using:

- **Accuracy:** Overall correctness of predictions
- **Precision and Recall:** To evaluate classification quality
- **Confusion Matrix:** To analyze misclassification between movement classes

4.7 Model Deployment

After training, the system was deployed as a real-time pipeline:

- **Real-Time Signal Processing:**
EMG signals are continuously read from Arduino through Shimmer sensor and processed in Python
- **Game Integration:**
Classified outputs are converted into commands and sent to the Unity game
- **End-to-End Flow:**
EMG Sensor(Shimmer) → web socket → Python Processing → AI Classification → Game Control

This enables real-time, hands-free interaction for rehabilitation and gaming.

Chapter 5

System Testing & Deployment

5.1 Testing Approach (Manual, Automated)

The testing approach for this system primarily consists of manual testing, supported by limited automated validation. Manual testing was conducted to simulate real-time user interaction with the EMG-based game, ensuring proper signal capture, processing, and control response. Automated testing was applied in specific areas such as validating model predictions and data processing. This approach ensures reliable system performance in real-world scenarios.

5.2 Unit Testing

Unit tests are implemented to validate the functionality of individual components, such as the model, Emg sensor, and python processing. These tests ensure that each component performs as expected before integration into the overall system.

Test ID	Component	Test Scenario	Expected Result
UT-01	Emg Sensor	Capture signal during muscle flex	Signal values change correctly
UT-02	Web socket	Read the Data	Communication
UT-03	Python processing	Normalize Signal	Output values scaled properly
UT-04	AI Model	Classify Signal	Correct movement prediction

Table 5.2: Unit Testing

5.3 Integration Testing

Integration testing verified communication between modules such as sensor, web socket, AI model, and game.

Test ID	Modules	Test Scenario	Expected Result
IT-01	EMG+Web socket	Signal transmission	Data Sent Correctly
IT-02	Web socket+Model	communication	Data received Without loss
IT-03	Game+Model	Movement prediction	Correct Game Response

Table 5.3: Integration Testing

5.4 User Acceptance Testing (UAT)

User acceptance testing will be conducted by a select group of users who will test the system in real-world scenarios. The goal is to ensure the game meets user expectations, is intuitive, and provides moves by accurately recognizing the signal from the model through web socket.

Test ID	User Task	Success Metric	Feedback
UAT-01	Play game using EMG	Perform therapy movement	Positive
UAT-02	Perform therapy movement	Accurate Detection	Satisfactory

Table 5.5: User Acceptance Testing (UAT)

5.5 Performance Testing

Performance testing focused on system responsiveness and real-time processing.

Test ID	Metric	Test Condition	Target
PT-01	Response time	EMG input = game reaction	< 1 second
PT-02	Stability	Continuous use	No crashes

Table 5.6: Performance Testing

5.6 Deployment Strategy

The system is deployed as a local setup, integrating hardware (EMG sensor), AI processing in Python, web socket integration and a Unity-based game. The deployment focuses on real-time interaction rather than cloud-based distribution.

5.7 Deployment Environment

The system operates in a local environment with:

- Python handling signal processing and classification
- A bridge for signal forwarding(Web socket)
- Unity game running on a PC

This setup ensures low latency and reliable real-time performance.

Chapter 6

Results and Discussion

6.1 Evaluation and Results

The system was evaluated based on multiple factors, including accuracy of EMG signal classification, real-time responsiveness, system stability, and user interaction. The evaluation focuses on how effectively the system converts muscle signals into game control actions while maintaining low latency and smooth performance.

During development, two different data acquisition approaches were implemented and analyzed to improve system efficiency and responsiveness.

6.1.1 Comparison with Initial Objectives

The system successfully achieved its core objectives of capturing EMG signals, processing them using an AI-based model, and converting them into real-time game control actions.

Initially, EMG data was collected using a Shimmer module, which enabled reliable signal acquisition but introduced noticeable delay during signal transmission and processing. To overcome this limitation, an alternative approach was implemented using a direct Python-based API, which bypassed the Shimmer module and significantly reduced latency. Additionally, the earlier use of an Arduino-based setup was replaced with a WebSocket-based communication architecture, enabling direct and efficient data transfer between the signal processing module and the Unity game.

This transition improved system stability and eliminated issues such as hardware overheating and connection interruptions. These improvements align with the project's objective of creating a responsive, real-time, and efficient EMG-based control system for rehabilitation.

6.1.2 User Feedback and Usability Testing

User feedback indicated that the system is generally easy to use and understandable after a short introduction. Most users were able to control the game using muscle movements without major difficulty. However, some users faced minor issues during initial use, mainly due to differences in muscle strength and signal consistency. A few users suggested improving the calibration process and adding more game features to make the experience more engaging. Overall, the system is usable, but it can be improved further to make it more consistent for different users.

6.1.3 Performance Evaluation

The system performance improved after changes were made in the architecture. Initially, the use of the Shimmer module caused delays in signal processing, which affected real-time response. After switching to a Python-based API approach, the delay was reduced, and signal handling became faster. Replacing the Arduino with a WebSocket-based communication system further improved performance by enabling direct and continuous data transfer between the processing module and the game. As a result, the system now responds more quickly to muscle movements, although slight delays may still occur in some cases.

6.1.4 Challenges and Limitations

During development, several challenges were faced. EMG signals are naturally noisy and vary from person to person, which makes consistent classification difficult. The initial use of the Shimmer module introduced delays, and the Arduino setup created issues such as heating and unstable connectivity. These problems required changes in the system design. Another

limitation is that the dataset was collected in a limited environment, which may affect performance in different conditions. Additionally, users may need calibration before using the system effectively. Despite these challenges, the system works as expected with some limitations.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The developed system, NeuroPlay ;AI-Driven EMG Gaming for Therapy, demonstrates the use of EMG signals for controlling a game through muscle activity. The system captures signals from biceps and triceps muscles, processes them using a machine learning model, and converts them into control actions in a Unity-based game.

During development, improvements were made by shifting from a Shimmer-based setup to a Python-based API approach, which reduced delay in signal processing. The replacement of Arduino with a WebSocket-based communication system also helped improve real-time interaction and system stability.

The system is able to perform basic muscle movement detection and allows users to interact with the game without traditional input devices. It provides a simple and engaging way to perform therapy-related exercises. However, the system still depends on proper signal quality and user calibration, and its performance can vary across different users

7.2 Future Enhancements

There are several areas where the system can be improved:

- **Wireless EMG Setup:**
Replacing wired connections with wireless sensors can improve usability and user comfort
- **Improved Signal Processing:**
Applying better filtering and feature extraction techniques can increase accuracy
- **Advanced Machine Learning Models:**
Using more robust models can help handle variations in EMG signals across users
- **Multi-Muscle Support:**
Extending the system to include more muscles can enable more complex controls like for hamstring muscle.
- **Game Improvements:**
Adding more levels and interactive features can improve user engagement
- **Session Data Storage:**
Implementing a system to store therapy session data, including EMG signals and performance metrics, can help track user progress over time and support analysis
- **Mobile or Portable System:**
Developing a mobile-based version can make the system more accessible

Bibliography

- Oskoei, M. A., & Hu, H. (2007). *Myoelectric control systems A survey. Biomedical Signal Processing and Control*, 2(4), 275–294.
- Phinyomark, A., Limsakul, C., & Phukpattaranont, P. (2012). *A novel feature extraction for robust EMG pattern recognition. Journal of Computing*, 1–7.
- Englehart, K., & Hudgins, B. (2003). *A robust, real-time control scheme for multifunction myoelectric control. IEEE Transactions on Biomedical Engineering*, 50(7), 848–854.
- Scheme, E., & Englehart, K. (2011). *Electromyogram pattern recognition for control of powered upper-limb prostheses: State of the art and challenges for clinical use. Journal of Rehabilitation Research and Development*, 48(6), 643–660.
- Farina, D., Jiang, N., Rehbaum, H., et al. (2014). *The extraction of neural information from the surface EMG for the control of upper-limb prostheses: Emerging avenues and challenges. IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 22(4), 797–809.
- Atzori, M., Gijsberts, A., Castellini, C., et al. (2014). *Electromyography data for non-invasive naturally-controlled robotic hand prostheses. Scientific Data*, 1, 140053.